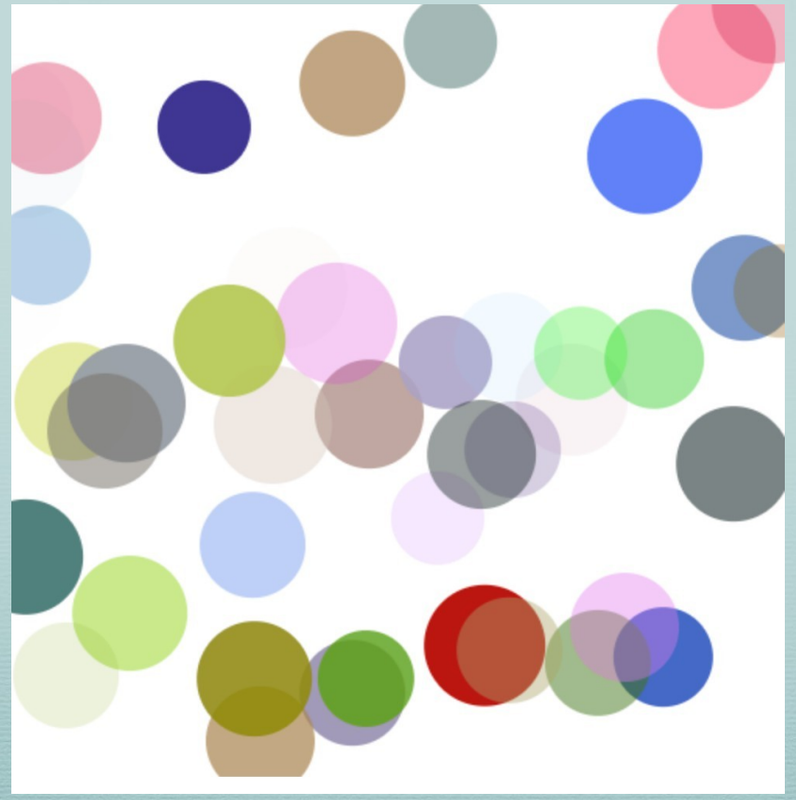
Advanced Web Graphics

source:https://www.w3schools.com

Bouncing Ball using Class



```
<body>
<canvas id="myCanvas" width="500" height="500"></canvas>
```

給予 500X500 的畫布

```
<script>
        // 見下頁
</script>

</body>
```

```
class Ball {
        constructor(x,y,r,colo
          r){ this.x=x;
          this.y=
          y; this.r
          = r;
          this.col
          or =
          color;
          this.dx = random(-
          3,3)+0.001; this.dy =
          random(-3,3)+0.001;
        }
        draw(g)
          { g.beginPa
          th();
          g.arc(this.x,this.y,this.r,0,Math.P
          I*2); g.fillStyle=this.color;
          g.fill();
        }
        move(){
          this.x = this.x + this.dx;
          if (this.x > width || this.x < 0 )
          {
            this.dx = -this.dx;
            this.x = this.x +
            this.dx;
          }
          this.y = this.y +
          this.dy;
          if (this.y > height ||
```

設定球的 x,y 位置，半徑 (r) ，與顏色 (color)，與球的水平 (dx) 垂直 (dy) 移動量

想想看，為什麼要有正負設計？

球的繪製，由球的 x,y 位置，半徑，與顏色決定球的外觀

計算球的移動，當碰到畫布 (canvas) 邊緣時，球會反彈，進行反向運動 (dx -> -dx, dy -> -dy)

```
var canvas = document.getElementById("myCanvas");
var ctx = canvas.getContext("2d");

var width = canvas.width;
var height = canvas.height;

function random(min,max)
{
      return Math.random()*(max-min)+min;
 }

var balls=[];
for( i = 0; i < 50; i++)
{                          隨機產生球的位置 (0~width, 0~height) 與半徑 (30+0~10)
      x=random(0,width); y=random(0,height);
      r = 30 + random(0,10);                                  產生隨機 RGBA 顏色，範圍從暗到亮 (0 ～ 255)
      color="rgba("+random(0,255)+","+random(0,255)+','+random(0,255)+","+Math.ran
      balls.push(new Ball(x,y,r,color));  從 balls 陣列尾端新增一棵球 (Ball 元素 )
 }
render();將所有球 (balls) 畫在畫布裡，是一個 recursive function,  會不斷的呼叫自己而達成動畫影格的繪製
```

```
function render(){
        ctx.clearRect(0,0, width, height);
        for (i =0; i< balls.length;i++) {
            balls[i].draw(ctx);
            balls[i].move();
        }
        window.requestAnimationFrame(re
nder);
}
```

清除上一影格的畫面，**避免畫面殘留的問題**

將每一個球畫出，並進行下一步的移動

約 1/60 秒後會呼叫 render ，**進行下一個畫面的繪製**，而形成動畫

LAB ：

1. 增加球的數量
2. 增加球與球之間的大小差異
3. 讓球碰觸到邊線即反彈

EventListener

element.addEventListener(event, function);

mouseover mouseout 事件

click

```
<button id="myBtn">Try it</button>

<script>
document.getElementById("myBtn").addEventListener("click",
myFunction);

function myFunction() {
  alert ("Hello World!");
}
</script>
```

addEventListener 可以監控事件 (click, mouseout…) ，若發生可以執行 myFunction()

Keyboard events

- key events
  - keydown
  - keyup

- <button id="myBtn">Try it</button>

- <script>
- document.getElementById("myBtn").addEventListener("keydown", myFunction);

- function myFunction() {
- alert ("Hello World!");
- }

2D Transformation

Outlines

- Geometric Transformation
  - Linear Transformation
    - Standard Basic
    - Transformation Composite
    - Transformation Homogenous

Linear Transformation

$$T(\alpha_0 \cdot u_0 + \ \ldots \ + \alpha_k \cdot u_k) = \alpha_0 \cdot T(u_0) + \ \ldots \ + \alpha_k \cdot T(u_k)$$

where $\alpha_0 \ldots \alpha_k$ are scalars

an

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot (\alpha_0 \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \alpha_1 \begin{bmatrix} x_1 \\ y_1 \end{bmatrix})$$

$$\Longrightarrow \alpha_0 \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \alpha_1 \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

2D Geometric Transformation

- Translatio
- n
- Rotation,
- Scaling,

  Sheer

2D transformation as Matrix

- Standard transformation used in computer graphics
  - **Translation** $T_{(d_x,d_y)}(P) = I \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} d_x \\ d_y \end{bmatrix}$

Non-linear

  - **Rotation** $R_\theta(P) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$
  - **Scaling** $S_{s_x,s_y}(P) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$
  - **Shearing** $Shear_{s_x}(P) = \begin{bmatrix} 1 & 0 \\ s_x & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$, $Shear_{s_y}(P) = \begin{bmatrix} 1 & s_y \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$
  - **All the above transformation are invertible (have inverse)**
    - $T^{-1}(T(P)) = T^{-1} \cdot T(P) = I \cdot P = P$

Transformation Linearity

- Basic standard transformation such as
  - scale,
  - rotate,
  - and shear
  - are linear transformation

Composite Transformation

- Composite transformation used in computer graphics
- $T_1(T_2(\ldots T_n(P)))$
- Translation cannot be expressed as a linear transformation and is therefore impossible to express it as a matrix multiplication.
- Need to extend 2D coordinate to 3D homogenous coordinate

Homogeneous Coordinate

- Represent coordinate in 2D dimension with 3-vector

$$(x,\ y) \implies (x,\ y,\ w)$$

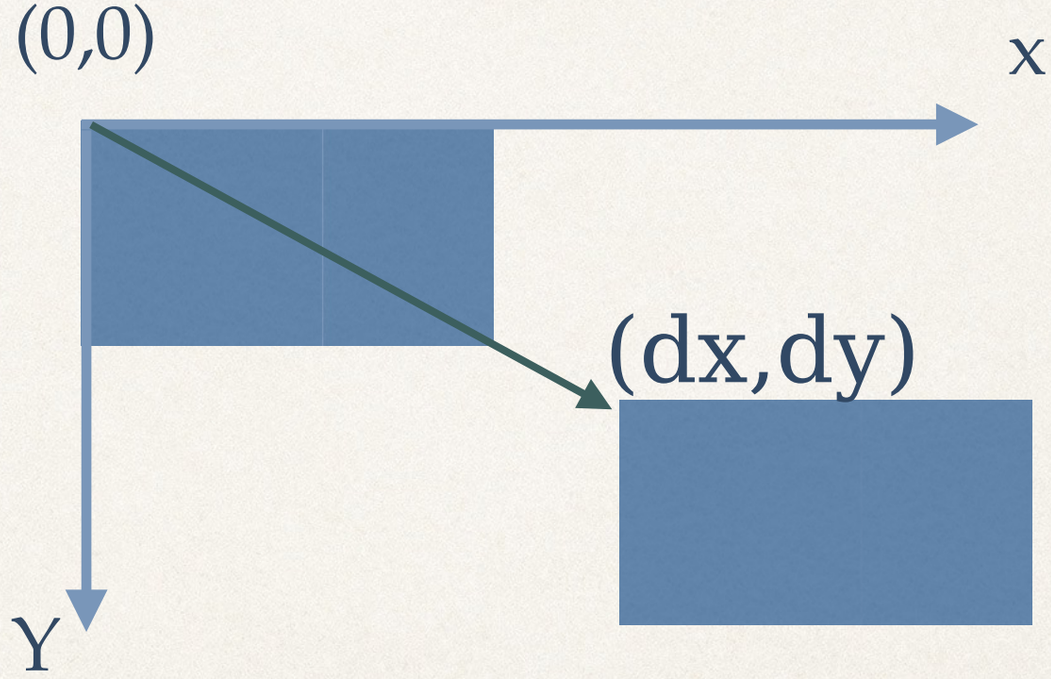
- Add a 3rd coordinate to every 2D point
  - $P = (x, y) \implies (x, y, 1)$

$$\begin{bmatrix} x \\ y \end{bmatrix} \implies \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

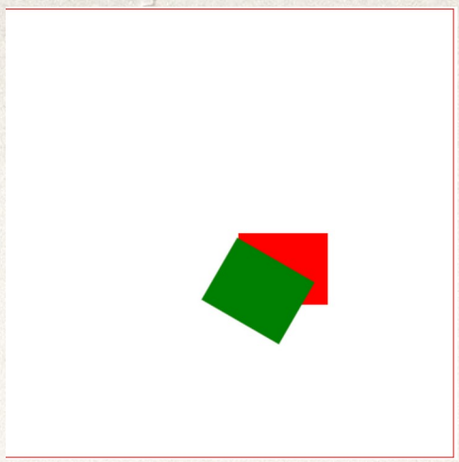
- Add a 3rd coordinate to every 2D vector
  - $\vec{v} = (x, y) \implies (x, y, 0)$

Translation(dx,dy)

$$\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}$$

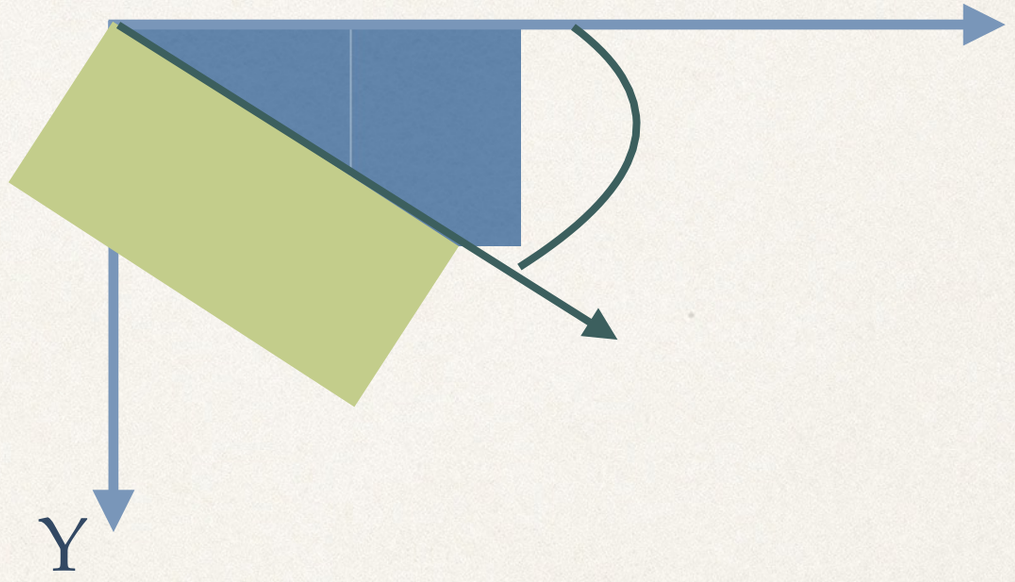


Rotation()



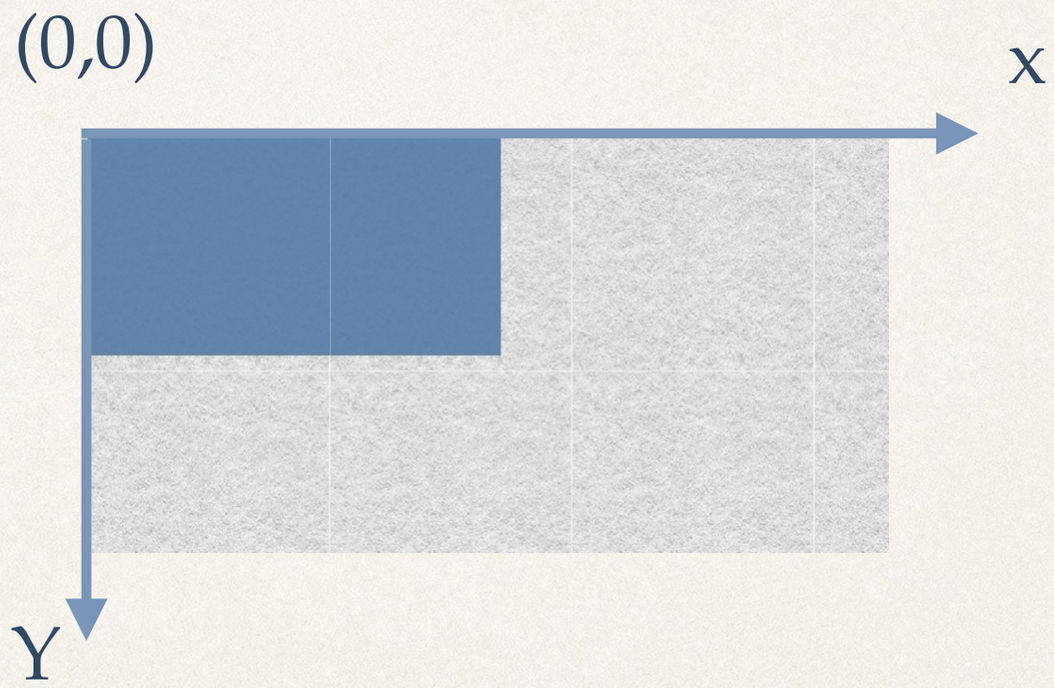
$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + ycos\theta \\ 1 \end{bmatrix}$$

```
<canvas id="myCanvas" width="500"
height="500"></canvas>
<script>
   canvas = document.getElementById("myCanvas");
   g = canvas.getContext("2d");
   g.translate(100,100);
   g.fillStyle="red";
   g.fillRect(0,0,100,80);
   g.rotate(Math.PI/6);
   g.fillStyle="green";
   g.fillRect(0,0,100,80);
</script>
```



Scale

$$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x \times s_x \\ y \times s_y \\ 1 \end{bmatrix}$$



Reflection

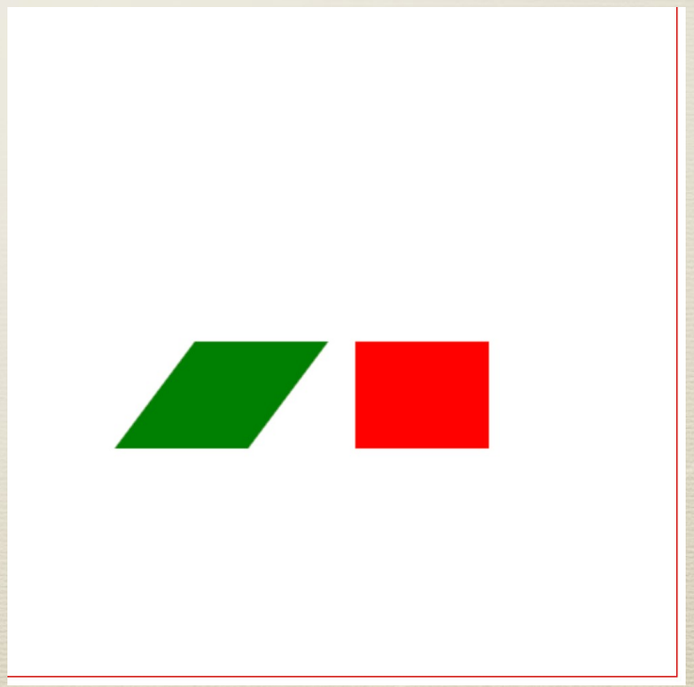
$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} -x \\ y \\ 1 \end{bmatrix}$$

```
canvas =
 document.getElementById("myCcanvas");
g = canvas.getContext('2d');

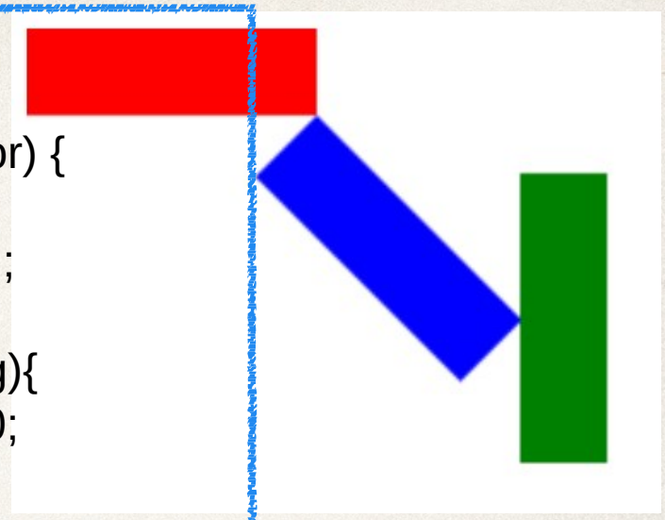
  g.translate(100,100);
  g.fillStyle="red";
  g.fillRect(0,0,100,80);
  g.scale(-1,1);
  g.fillStyle="green";
  g.fillRect(0,0,100,80);
```

Sheer

$$\begin{bmatrix} 1 & c_x & 0 \\ c_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + y \times c_x \\ y + x \times c_y \\ 1 \end{bmatrix}$$



```
<!DOCTYPE html>
<html><head><meta charset="utf-8"></ head><body>
<!-- canvas 的定義 -->
<canvas id="a_canvas" width="300" height="300"></canvas>
<script type="text/javascript">
 //  取得
 canvas var
 canvas =
document.getElementById("a_canvas");
 //  取得用來描繪的上下文
 var ctx =
canvas.getContext("2d");
 //  定義描繪矩形的方法
function drawRect(context, color) {
    context.fillStyle = color;
    context.fillRect(0, 0, 100, 30);
}
function rotateDeg(context, deg){
    var rad = deg * Math.PI / 180;
    context.rotate(rad);
}
drawRect(ctx, "red");
ctx.save(); //  儲存狀態
ctx.translate(100,30); //  平行移動位置
rotateDeg(ctx, 45); //  旋轉 45 度
drawRect(ctx, "blue");
ctx.restore();  //  復原狀態
// (3) 指定移動與旋轉並描繪
ctx.save(); //  儲存狀態
ctx.translate(200,50); //  平行移動位置
rotateDeg(ctx, 90); //  旋轉 90 度
drawRect(ctx, "green");
ctx.restore();  //  復原狀態
</ script>
</ body></ html>
```



Transformation Sequence & Scene graph

Math: $T_1(T_2(T_3(Shape_1)))$

Matrix: $T_1 \times T_2 \times T_3 \times Shape_1$

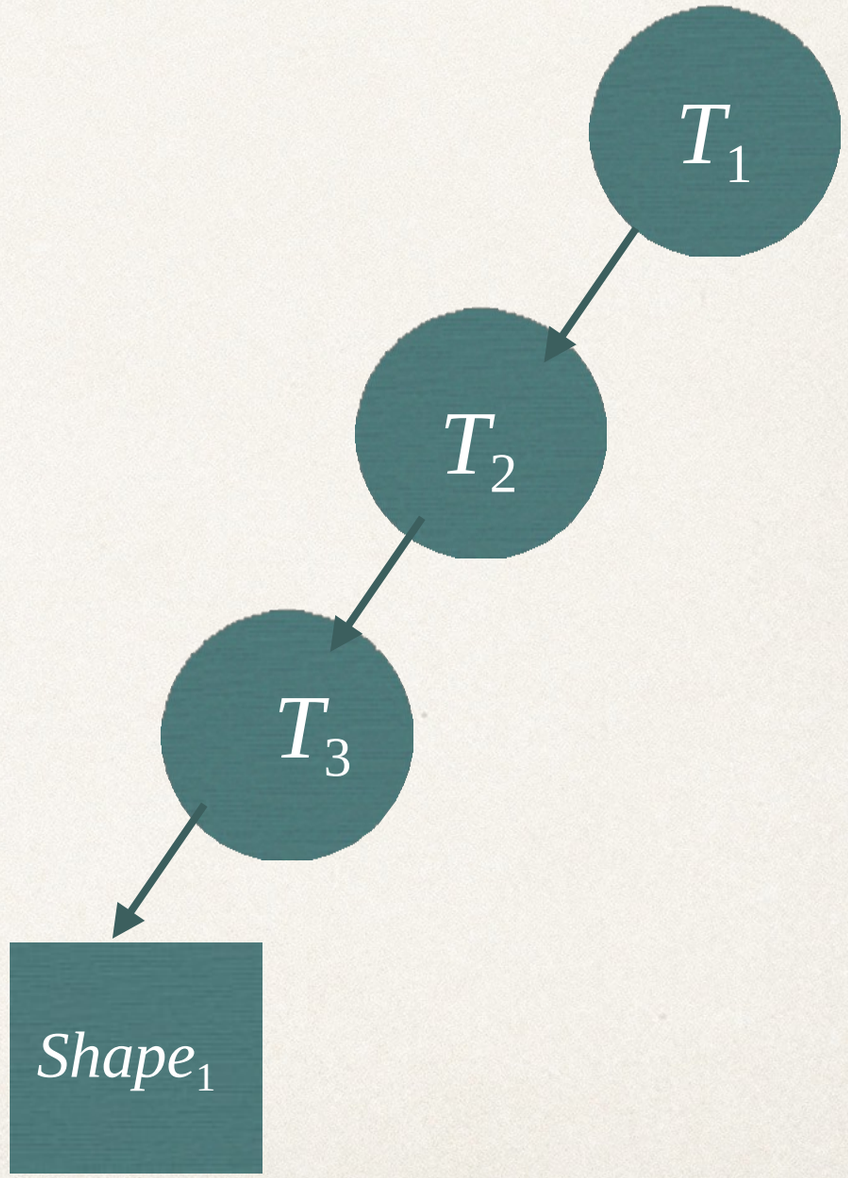
$Shape_1$

T_1

T_2

T_3

$Shape_1$



save() and restore() default state

Math: $T_1(T_2(T_3(Shape_1))$

Matrix: $T_1 \times T_2 \times T_3 \times Shape_1$

save()

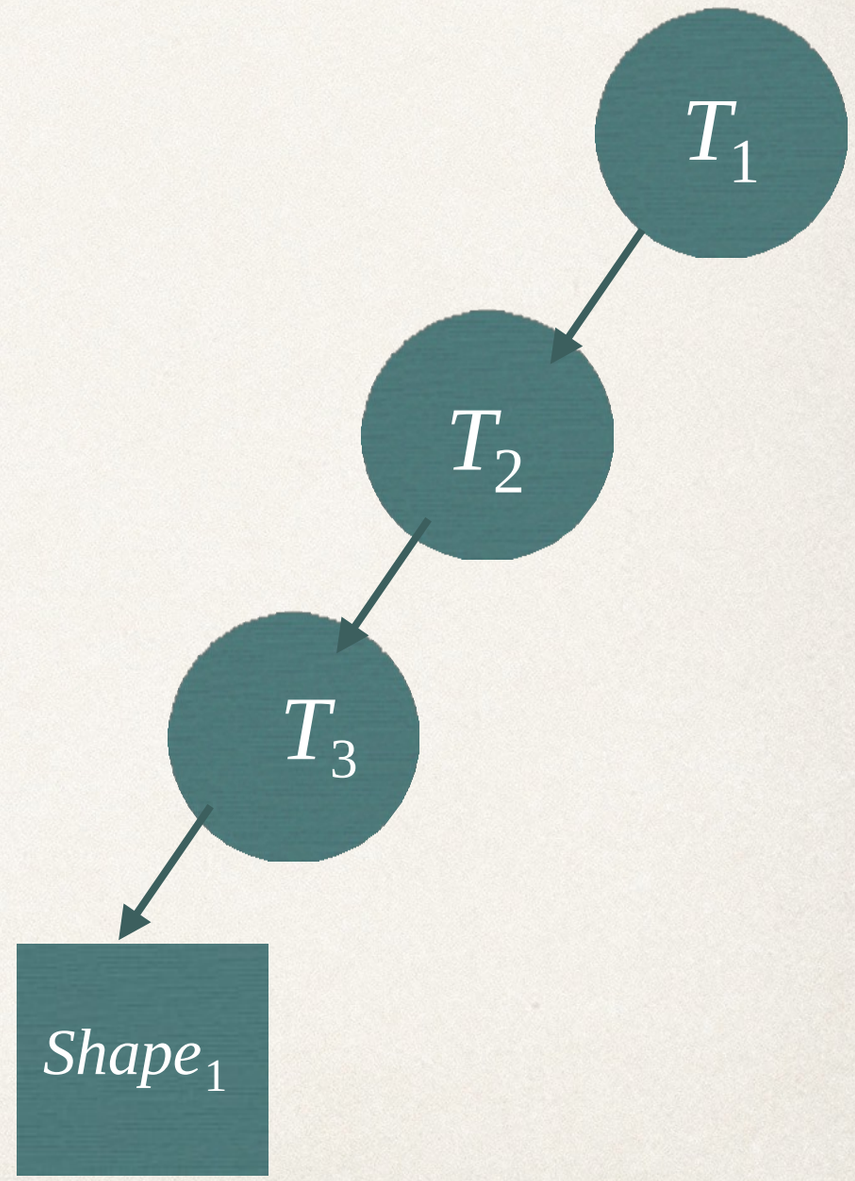
T_1

T_2

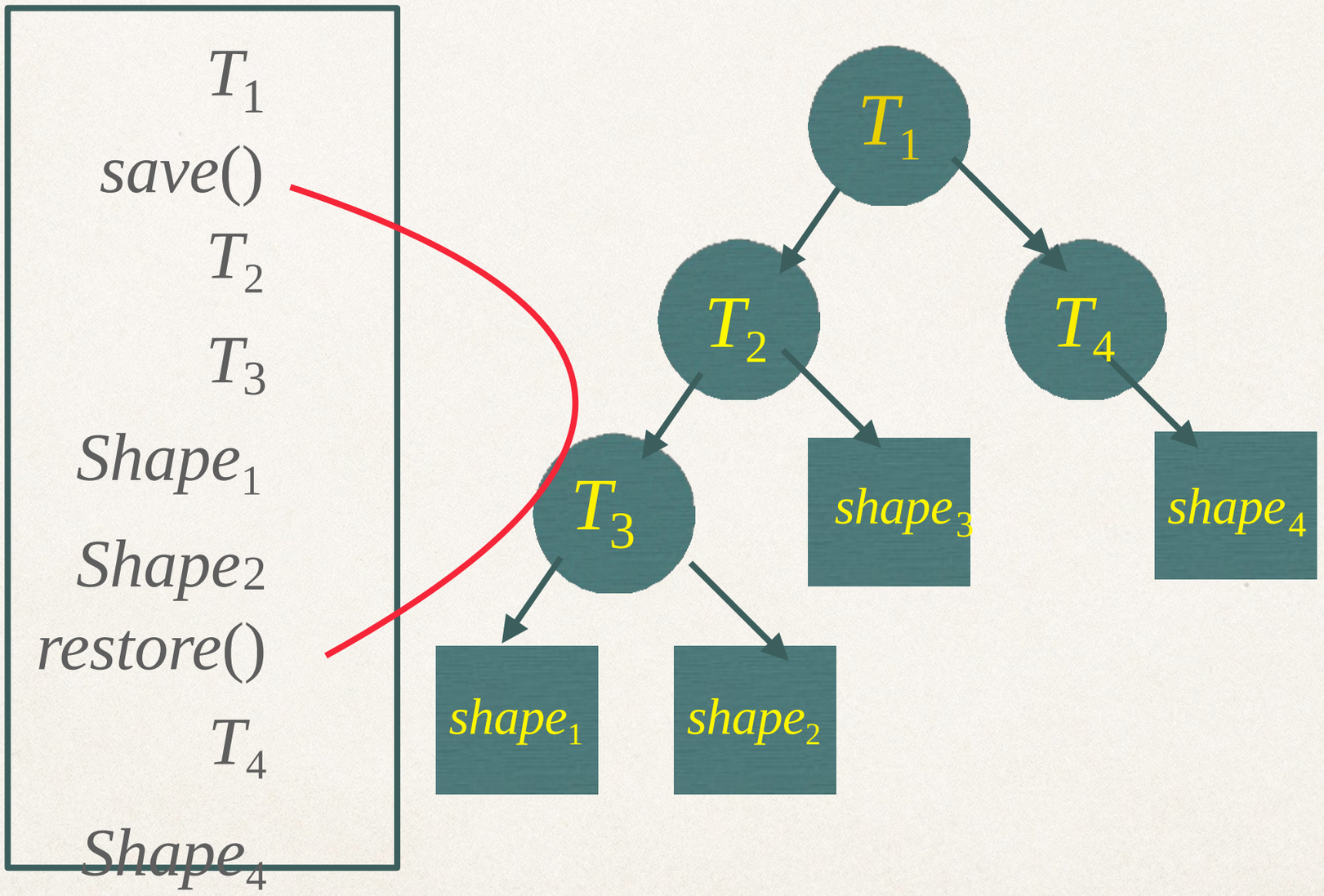
T_3

$Shape_1$

restore()



Transformation Sequence & Scene graph



Digital Clock

Clock

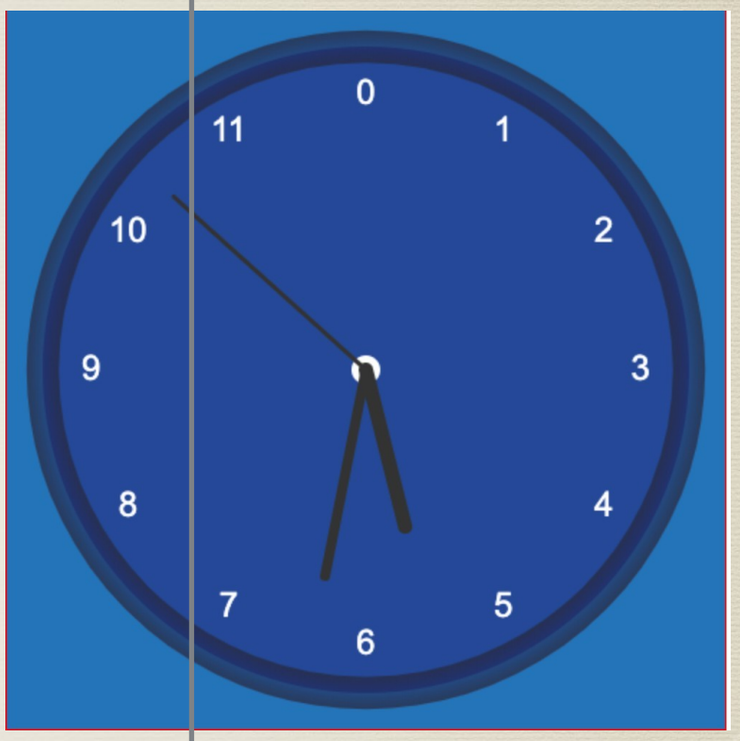
```
var canvas;
var g;
var radius;

function init(){
    canvas =                document.getElementById("canvas");
    g = canvas.getContext('2d');
    radius = canvas.height/2*0.9;
    setInterval(drawClock,1000);
}
```

設定 drawClock 呼叫間隔，每 1sec 重畫時間

```
function drawClock(){
    g.save();
    g.clearRect(0,0,canvas.width,canvas.height);
    g.translate(canvas.width/2,canvas.height/2);
    drawFace();
    drawText();
    drawTime();
    g.restore();
}
```

畫 clock，儲存狀態 -> 移到 clock 中心 -> 畫出 clock(drawFace, drawText, drawTime)

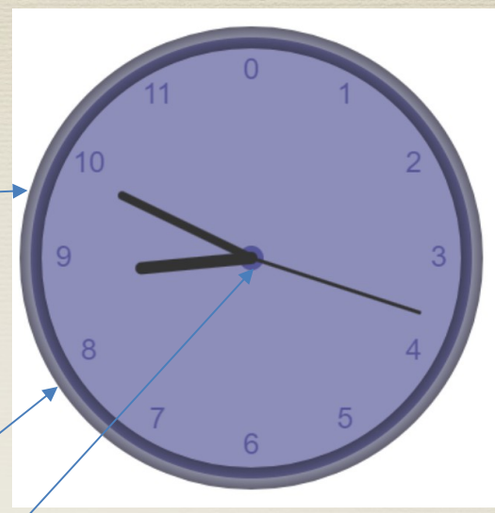


```
function drawFace(){
    g.beginPath();
    // outer cicle
    g.arc(0,0,radius,0,2*Math.PI);
    g.fillStyle='rgb(30,30,120,0.5)';
    g.fill();

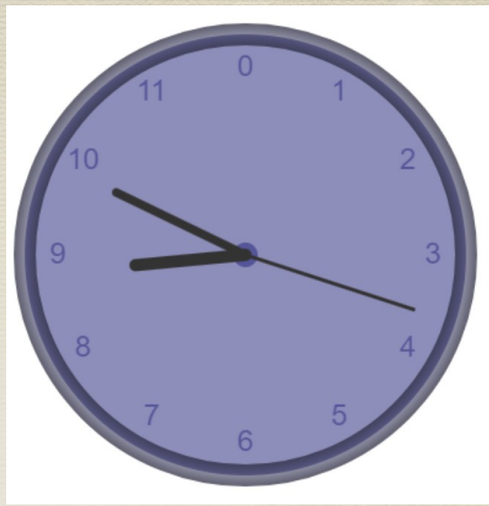
    grad = g.createRadialGradient(0,0,radius*0.90, 0,0,radius*1.1);
    grad.addColorStop(0, '#333');
    grad.addColorStop(0.5, 'rgb(30,30,70,0.5)');
    grad.addColorStop(1, '#333');
    g.strokeStyle = grad;
    g.lineWidth = radius * 0.1;
    g.stroke();

    g.beginPath();
    //inner circle
    g.arc(0,0,10,0,2*Math.PI);
    g.fillStyle = '#fff ';
    g.fill();
}
```

畫 clock 的外觀



```
function drawText(){
        g.font = 24 + "px arial";
        g.textBaseline="middle";
        g.textAlign="center";
        for(i=0 ; i< 12;i++){
            g.save();
            g.rotate(Math.PI/6*i);
            g.translate(0,-radius*0.85);
            //g.rotate(-Math.PI/6*i);
            g.fillText(''+i,0,0);
            g.restore();
        }
}
```



畫 clock 0 ～ 11 的數字，共 12 格，所以每格角度為 PI/6

想想看：為什麼是先 rotate 再 translate? 若順序相反會發生什麼狀況？

```
function drawTime(){                    畫 clock 的秒針、分針、時針
    var date = new Date();

    seconds = date.getSeconds()*6;      360/60
    minites = (date.getMinutes()+ date.getSeconds()/60)*6;    360/60
    hours = (date.getHours()+(date.getMinutes()+ date.getSeconds()/60)/60) %12*30;
                                        將 minites 除 60 ，轉為小時      轉 12 小時制，每小時 30 度
    drawHand(seconds,radius*0.8,3);
    drawHand(minites,radius*0.65,7);
    drawHand(hours,radius*0.5,10);
}
function drawHand(angle,length,width){
            g.save();
        g.beginPath();
        g.lineCap='round';
        g.lineWidth=width;
        g.rotate(angle*Math.PI/180);
        g.moveTo(0, 0);
        g.lineTo(0, -length);
        g.stroke();
        g.restore();
      }
```

Firework

Source: Coding Fireworks

```
<html>
<body>
<canvas id="fireworks" width="500" height="500" style="border:1px solid #000000;"></canvas>
    <script>
        canvas = document.getElementById("fireworks");
        const ctx = canvas.getContext('2d');
        // firework JavaScript will go here
    </script>
</body>
</html>
```

Drawing Fireworks

- The Firework class will represent a single firework rocket launching into the sky
- The Particle class will represent the particles that are emitted when the firework explodes



```
class Particle {
constructor(x, y, color) {
this.x = x;
this.y = y;
this.color = color;
this.vx = (Math.random() - 0.5) * 8;
this.vy = (Math.random() - 0.5) * 8;
this.alpha = 1;
this.friction = 0.99;
}
```

```
draw() {
ctx.globalAlpha = this.alpha;
ctx.beginPath();
ctx.arc(this.x, this.y, 2, 0, Math.PI * 2);
ctx.fillStyle = this.color;
ctx.fill();
}
```

如果使用 stroke()?

```
ctx.strokeStyle = "red";
ctx.stroke();
```

```
update() {
this.vx *= this.friction;
this.vy *= this.friction;
this.x += this.vx;
this.y += this.vy;
this.alpha -= 0.01;
}
}
```

- The Particle class is defined to represent individual particles in a firework explosion.
- It has properties for its position (x, y), color, velocity, opacity (alpha), and friction.
- The draw method is responsible for drawing an individual particle on the canvas. Draw particle by arc().
- The update method changes the velocity and position of the particle as well as reduces its opacity, simulating the particle flying out and fading away.

```
class Firework {
constructor(x, y, color) {
this.x = x;
this.y = y;
this.color = color;
this.vx = 0;
this.vy = Math.random() * -2.5 - 0.5;
this.particles = [];
this.lifespan = 180;
this.hasExploded = false;
}

draw() {
ctx.beginPath();
ctx.arc(this.x, this.y, 3, 0, Math.PI * 2);
ctx.fillStyle = this.color;
ctx.fill();
}

explode() {
for (var i = 0; i < 50; i++) {
this.particles.push(new Particle(this.x, this.y, this.color));
}
}

update() {
this.lifespan--;

if (this.lifespan <= 0 && !this.hasExploded){
this.explode();
this.vx = 0;   this.vy = 0;
this.hasExploded = true;
} else if (this.lifespan > 0) {
this.y += this.vy;
}

for (var i = 0; i < this.particles.length; i++) {
this.particles[i].update();
this.particles[i].draw();
}
}
}
```

- The Firework class is defined to represent a firework rocket before and after explosion.
- It has properties for its position (x, y), color, velocity, particles array (will contain instances of Particle after explosion), lifespan (how long before it explodes), and hasExploded (a flag indicating if it has exploded).
- The draw method draws the firework rocket on the canvas.
- The explode method creates multiple particles at the firework's current position, simulating an explosion.
- The update method updates the position of the firework or triggers the explosion when the lifespan is over. It also updates and draws particles if the firework has exploded.

```
fireworks = [];

function animate() {
ctx.fillStyle = 'rgba(0, 0, 0, 0.1)';
ctx.fillRect(0, 0, canvas.width, canvas.height);

for(var i = 0; i < fireworks.length; i++){
fireworks[i].update();
fireworks[i].draw();
if (fireworks[i].lifespan <= 0 && fireworks[i].particles[0].alpha <= 0) {
fireworks.splice(i, 1);
}
};

if (Math.random() < 0.015) {
const x = Math.random() * canvas.width;
const color = 'rgb('+Math.random()*255+','+Math.random()*255+','+Math.random()*255+')';
fireworks.push(new Firework(x, canvas.height, color));
}
requestAnimationFrame(animate);
}

animate();
```

- The animate function is the core of the animation. It uses requestAnimationFrame to create a loop that updates the canvas drawing in quick succession, creating the animation effect.
- Within this function, the canvas is cleared every frame to make space for the new positions of fireworks and particles.
- New instances of Firework are occasionally pushed to the fireworks array to launch new rockets.
- Each firework in the fireworks array is updated and drawn every frame. If the firework has exploded and all particles have faded away, it is removed from the array.

Add gravity force

- **Newton dynamics f = ma**

- **Vectors, velocity, and acceleration:**
  - v(n + 1) = v(n) + a * t
  - p(n + 1) = p(n) + v * t

- //ex. particle update func.
- update() {
- this.vx *= this.friction;
- this.vy *= this.friction;
- this.x += this.vx;
- this.y += this.vy;
- this.alpha -= 0.01;
- }

參考左邊公式，加入加速度

Homework_1(demo later with Homework_2)

* Design particle effects such as fireworks, particles using canvas drawing.
* Particle can be any shape like circle, star, ..
* Path of particles can be affected by gravity or other force
* Add forest as your background like lec03_Canvas for the Nature I.pptx
* Please cite the source of reference